A Two-Path Resilience Architecture for Attestation-Triggered Network Isolation and SLM-Assisted Remediation in Connected Vehicles

Prajwal Shinde, *2285327, Sapienza University of Rome*
Basanta Humagain, *2249217, Sapienza University of Rome*

Abstract—Modern connected vehicles utilize hardware-rooted Over-the-Air (OTA) update and attestation frameworks to secure firmware distribution across Electronic Control Units (ECUs). However, existing all-or-nothing attestation models omit availability guarantees; a single node failing its integrity check permanently immobilizes the entire vehicle network. This paper proposes a two-path resilience architecture that closes this availability gap without compromising underlying security baselines. A real-time, deterministic path reacts within microsecond-scale network deadlines to isolate a failing non-critical node at the gateway level and inject pre-compiled fallback frames, transitioning the vehicle into a secure limp-home mode. Crucially, any failure within safety-critical nodes bypasses this degradation path and defaults the vehicle directly to a secure locked state. Once isolated, an asynchronous, out-of-band path leverages a quantized Small Language Model (SLM) on the Telematics Control Unit to classify hardware telemetry logs and generate a structured remediation manifest, enabling the backend to deploy a targeted micro-patch instead of a full firmware re-flash. Critically, the SLM is strictly decoupled from the live control loop, ensuring that algorithmic errors have a bounded blast radius. We outline this multi-timescale architecture and present an evaluation methodology to validate its real-time latencies, security regression boundaries, and system availability properties.

Index Terms—Connected vehicle security, Over-the-Air updates, Trusted Platform Module, remote attestation, Denial of Service, CAN bus isolation, small language models, edge AI, graceful degradation

1 Introduction

Modern automotive systems are complex, distributed cyber-physical platforms coordinating up to one hundred Electronic Control Units (ECUs) over networked in-vehicle architectures such as the Controller Area Network (CAN) [12]. Because these vehicles are designed for service lives extending well beyond a decade, Over-the-Air (OTA) software updates have transitioned from a convenience feature to a regulatory mandate—specifically under UN Regulations No. 155 and No. 156—to patch zero-day vulnerabilities and maintain fleet-wide type approval [16]. Plappert and Fuchs established a secure cryptographic lifecycle for this pipeline by combining a Trusted Platform Module (TPM) 2.0-anchored symmetric update distribution framework [12] with a Device Identifier Composition Engine (DICE)-based post-update attestation scheme [11]. This dual-layer approach offloads heavy public-key operations from resource-constrained ECUs and cryptographically validates runtime firmware integrity before permitting normal vehicle operation.

While mathematically robust against firmware tampering, both frameworks explicitly exclude Denial of Service (DoS) vectors from their threat models [11]. This bounding leaves a critical system availability vulnerability. Because the baseline safety policy inhibits vehicle ignition until *every* node successfully attests, a single non-critical ECU attestation failure—whether caused by an active firmware hack, an intermittent memory glitch, or a transient hardware bit-flip—indefinitely suspends operational availability by locking the vehicle's drive mode. No mechanism currently exists to safely isolate the anomalous node or recover vehicle operations without a physical workshop intervention.

1.1 Problem Statement

We address the following core system availability challenge: *given a hardware-rooted TPM/DICE attestation failure for non-critical ECU, how can the in-vehicle network dynamically respond to preserve physical vehicle availability without weakening underlying safety-critical security guarantees, violating hard real-time network timing deadlines, or incurring massive full-image firmware re-flash overhead?*

This is fundamentally a mixed-criticality systems routing problem rather than a purely cryptographic one. The underlying attestation layer correctly *detects* a cryptographic deviation, but lacks the architectural infrastructure to isolate the failing node or diagnose its root cause out-of-band.

1.2 Contribution

To bridge this operational gap, we propose a **two-path resilience architecture** that decouples real-time physical

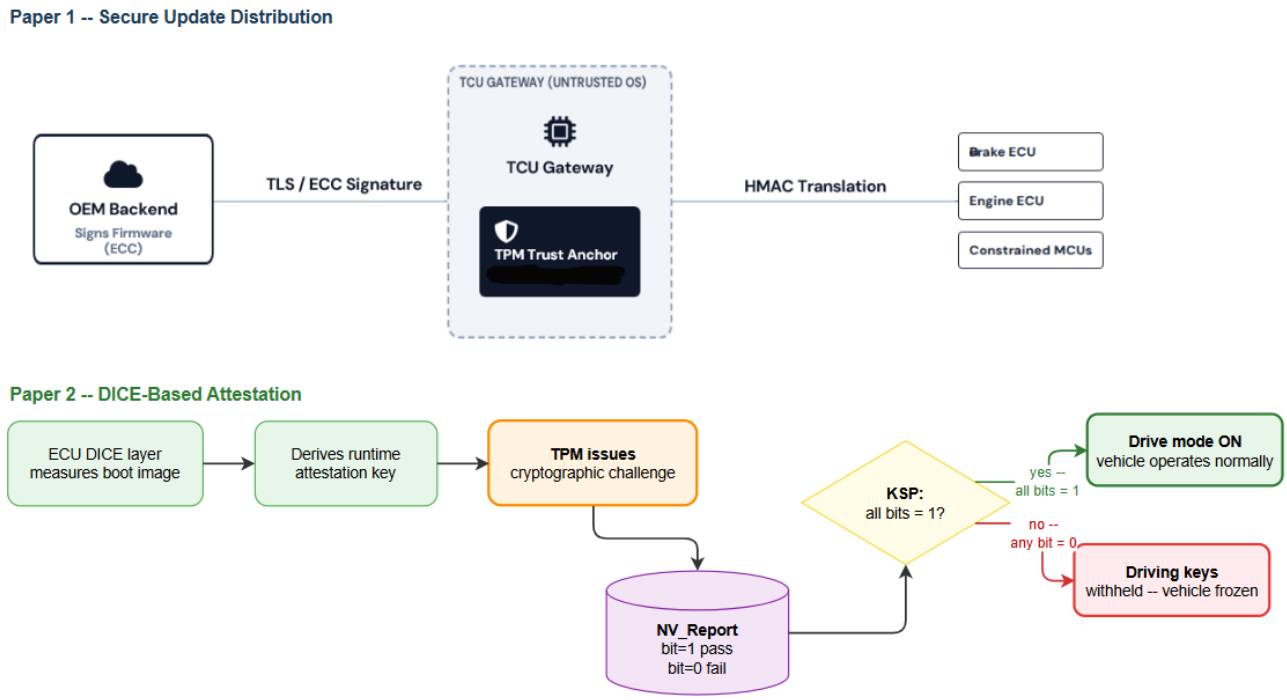

Fig. 1. System topology detailing the baseline update validation model from previous literature: the asymmetric-to-symmetric distribution pipeline (Paper 1) and the stationary, all-or-nothing NV_{Report} attestation gate (Paper 2).

safety from asynchronous post-failure diagnostics across distinct operational timescales:

Path 1 (Deterministic Real-Time Path): Executed at the microsecond scale within the central gateway, this purely programmatic path isolates the failed node's routing tables and substitutes its expected cyclic traffic with static, pre-compiled baseline frames, safely dropping the vehicle into a secure limp-home mode before network timeout deadlines expire.

Path 2 (Asynchronous Out-of-Band Path): Executed at the seconds scale within the Telematics Control Unit (TCU), this path utilizes a localized, resource-constrained Small Language Model (SLM) to evaluate hardware telemetry error vectors. It diagnoses the root cause of the fault and compiles a machine-readable manifest to coordinate an optimized, remote micro-patch rather than an expensive fleet-wide re-flash.

Crucially, our contribution is structural rather than cryptographic. By completely isolating the SLM from the active control loop and the TPM's core policy decisions, we achieve a safety-by-construction design.

2 Background

2.1 TPM-based Secure Update Distribution(Paper 1)

The baseline secure over-the-air (OTA) update framework established by Plappert and Fuchs utilizes a hardware-rooted Trusted Platform Module (TPM) 2.0 integrated within the Telematics Control Unit (TCU) [12], as diagrammed in the top half of Fig. 1. The TCU serves as the vehicle's edge gateway, bridging external public networks with internal domain buses. The TPM acts as an isolated security coprocessor, allowing the system to enforce cryptographic policies without trusting the TCU's primary general-purpose operating system.

As summarized in Fig. 1, this update distribution mechanism operates in two distinct phases:

- **Authenticated Distribution Phase:** The OEM cloud backend signs a firmware update manifest using heavy asymmetric keys. The in-vehicle TPM validates this signature internally and derives a localized, lightweight symmetric Hash-based Message Authentication Code (HMAC) key for each target Electronic Control Unit (ECU). This offloads public-key processing overhead from resource-constrained embedded automotive nodes.
- **Installation Authorization Phase:** The TPM evaluates an Enhanced Authorization (EA) policy engine. It natively unseals update decryption keys only when specific programmatic conditions—including system state flags, target component identity strings, and cryptographic version anti-rollback checks—are satisfied.

2.2 DICE-Based Post-Update Attestation (Paper 2)

To ensure the vehicle network is executing authentic firmware following a reboot cycle, the architecture incorporates a hardware-rooted Device Identifier Composition Engine (DICE) primitive into each participating ECU [11], shown in the bottom half of Fig. 1.

During a stationary post-update validation window, each ECU's DICE layer measures the loaded boot image to derive a unique runtime attestation key. If an image has been modified or tampered with, the derived key changes,

causing the node to fail the cryptographic challenge issued by the central TPM.

The TPM tracks these network-wide attestation outcomes within a dedicated, tamper-resistant non-volatile register index known as *NVReport* (NV_{Report}), where each node's status is logged as a binary bit (1 for success, 0 for failure). Under the baseline Key Sealing Policy (KSP), a rigid all-or-nothing condition is enforced: if any single bit within NV_{Report} is 0, the master driving keys are withheld, permanently freezing the vehicle network to prevent a compromised system from transitioning into normal drive mode.

3 Overview of Proposed Approach

We propose a **two-path resilience architecture** that extends the existing TPM/DICE attestation mechanism of Plappert and Fuchs [11] with an explicit, automated response to attestation failure. The design separates responsibilities by timescale: a deterministic *real-time path* (Section 3.2) operates entirely within the timing budget of the in-vehicle network, and an asynchronous *out-of-band path* (Section 3.3) operates only once that budget no longer applies. Both paths are grounded in a single formal extension to the TPM's existing policy engine, introduced first.

3.1 A split-key extension to the Key Sealing Policy

Paper 2 records the outcome of each ECU's attestation as a single bit in a protected non-volatile bit-field, $NV_{\text{Report}} \in \{0,1\}^N$, where N is the number of participating ECUs, and gates release of a single Driving Key via a Key Sealing Policy (KSP) that requires every bit to equal 1 [11]. We partition the index set $\{1,\dots,N\}$ into a safety-critical tier $\mathcal{C}$ (size M) and a non-critical tier $\mathcal{U}$ (size $N-M$), assigned by the OEM at provisioning time and authenticated with the same key already used to sign the update manifest in Paper 1. Writing NV_{R} for NV_{Report}, the original KSP condition is preserved unchanged as the policy for the *Full* driving key:

$$\text{Release } K_{\text{Full}} \iff \forall i \in \mathcal{C} \cup \mathcal{U},\ NV_{\text{R}}[i] = 1. \tag{1}$$

We add a second, independently sealed key, K_{Limp}, governed by a strictly weaker policy:

$$\text{Release } K_{\text{Limp}} \iff \Big(\forall i \in \mathcal{C},\ NV_{\text{R}}[i]=1\Big) \wedge \Big(\exists j \in \mathcal{U},\ NV_{\text{R}}[j]=0\Big). \tag{2}$$

Both conditions are expressible as a conjunction of `TPM2_PolicyNV` comparisons against the existing NV_{Report} index, combined with `TPM2_PolicyOR` to select between the two key handles – this requires only a provisioning-time policy configuration change and a second sealed key object, not new TPM hardware or firmware. No bit in NV_{Report} is ever written by software outside the existing DICE challenge-response exchange of Paper 2, so the safety property of the original design is preserved exactly: Eq. (1) can never evaluate true if any safety-critical ECU has failed, and K_{Limp} has strictly less authority than K_{Full} by construction, capped at whatever drive parameters the OEM provisions for the limp-home profile.

Crucially, if any safety-critical node within the tier $\mathcal{C}$ fails attestation, neither Eq. (1) nor Eq. (2) can evaluate to true. In this scenario, the vehicle immediately defaults to the baseline locked state, entirely inhibiting drive mode. This ensures that safety-critical isolation remains strictly out of scope for policy degradation, preserving the zero-tolerance hazard threshold of the original underlying architecture.

3.2 Path 1 – Deterministic Gateway Response

The Central Gateway (CGW) holds an open `TPM2_PolicySession` against NV_{Report} and is notified of a state change either by polling the relevant NV index over the internal SPI interface or via a hardware interrupt line from the TPM's GPIO, depending on the platform deployment. Because Plappert and Fuchs's prototype uses an SPI-attached Infineon Iridium9670 TPM, the polling interface requires no additional hardware footprint [7], [12].

Upon observing $NV_{\text{Report}}[j] = 0$ for an auxiliary or non-critical ECU $j \in \mathcal{U}$, the CGW bypasses software-layer inference and executes a hardware-reflex isolation sequence broken down into three deterministic steps:

1) **Network-Layer Isolation:** The CGW's network processor instantly clears j's configuration entries from its forwarding and routing registers. Consequently, any malicious or corrupted data frames addressed to or originating from j's Controller Area Network (CAN) identifier range are blocked at the gateway boundary and are no longer relayed onto adjacent domain buses.
2) **DMA-Driven Frame Substitution:** Concurrently with isolation, the CGW's Direct Memory Access (DMA) engine assumes control of the broadcast slots previously allocated to the silenced node. The DMA engine begins cycling a small, fixed *Baseline Safe-State Frame Array* directly from physical Read-Only Memory (ROM) onto the active domain transceivers. This array is pre-compiled at design time and validated once during system certification.
3) **Timeout Fault Mitigation:** The injected substitution frames perfectly mimic j's expected cyclic transmission periods and Cyclic Redundancy Checks (CRC), but carry static, default nominal values (e.g., a body-control module reporting zero speed-dependent state and a nominal status flag). This satisfies the message counters of healthy nodes depending on j, preventing them from raising a missing-message timeout fault.

Because both the routing-table modification and the frame substitution are implemented strictly as static table lookups and raw memory copies rather than an execution search procedure, this path is designed to complete within a sub-millisecond timeline ($< 50\,\mu$s). This operational latency is several orders of magnitude below the $\sim$10 ms timeout window typically tolerated by automotive CAN nodes before raising a critical system fault, allowing a seamless transition into the safe, degraded limp-home mode.

3.3 Path 2 – Out-of-Band SLM-Assisted Diagnosis

Once K_{Limp} has been released and the vehicle is operating safely in its degraded mode, Path 2 activates a quantized

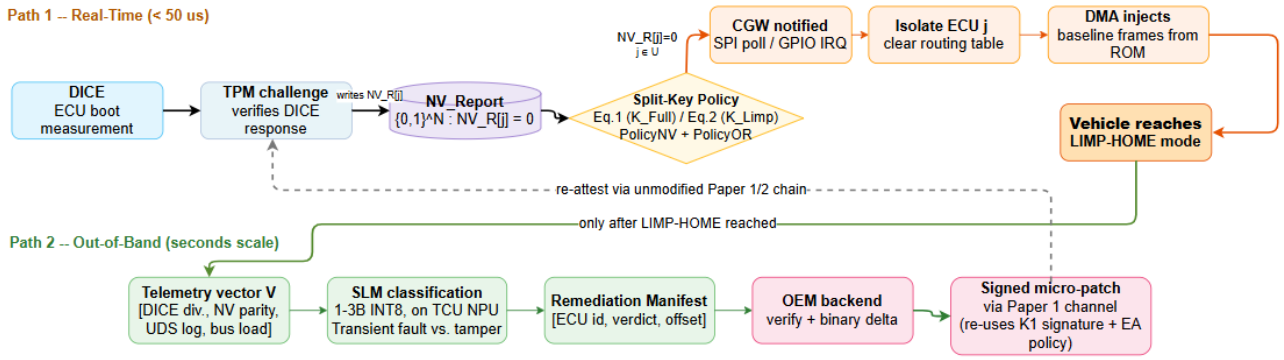

Fig. 2. Two-path resilience architecture. Path 1 is fully deterministic and operates within the in-vehicle network's reaction time budget, governed by the split-key policy of Eqs. (1)–(2). Path 2 activates only after K_{Limp} has been released and is strictly advisory: its output cannot bypass the unmodified authentication and authorization chain of Paper 1.

Small Language Model (SLM) in the 1–3 billion parameter range. The model executes at INT8 precision on an automotive-grade NPU integrated directly into the Telematics Control Unit (TCU).

Crucially, the SLM does not process unstructured natural language or free text. Instead, it consumes a fixed-length *Structured Hardware Telemetry Vector*:

$$\vec{V} = [\,\text{DICE divergence},\ \text{NV parity},\ \text{UDS encoded feature},\ \text{bus load}\,] \tag{3}$$

This vector is assembled directly from the failed ECU's diagnostic registers and the preceding window of Unified Diagnostic Services (UDS) network traffic, with read-only access strictly enforced at the hardware bus interface level. The UDS component is not an unstructured raw log, but a fixed-length encoded feature vector to remain mathematically compatible with the static dimensions of $\vec{V}$.

The SLM's task is limited to a bounded pattern classification problem over this vector, separating faults into two distinct profiles:

- **Transient Hardware Fault Profile:** Characterized by an isolated, single-bit divergence in non-volatile memory paired with an unremarkable, standard preceding UDS history.
- **Deliberate Tampering Profile:** Characterized by a contiguous, multi-byte memory divergence heavily correlated with anomalous preceding UDS network entries, such as unauthenticated Diagnostic Session Control (service `0x10`) or Communication Control (service `0x28`) requests.

The SLM's only output is a structured, schema-constrained *Remediation Manifest*. This file contains only the isolated node's identifier, a binary `TRANSIENT_FAULT` or `MALICIOUS_TAMPER` verdict, and, where applicable, the exact implicated memory offset range.

3.4 Why on-vehicle processing is necessary

A natural question is why this classification could not run at the OEM backend instead, given that the TCU already has cellular connectivity. The manifest's value lies specifically in the memory offset it reports: this is what allows the backend to generate a *micro-patch*, addressed to only the implicated region, rather than a full re-flash. Producing that offset requires parsing $\vec{V}$ at the point where it is generated; forwarding raw diagnostic vectors from an entire fleet for centralized parsing would reintroduce the bandwidth and backend-compute cost the manifest is designed to avoid, and connectivity cannot be assumed at the exact moment of failure, so on-vehicle processing also allows the manifest to be queued locally until upload is possible.

3.5 Authority boundary and remediation execution

The SLM cannot issue `IsolateECU` actions, cannot write to NV_{Report}, and cannot construct or satisfy a `PolicyNV` or `PolicyOR` assertion for either K_{Full} or K_{Limp}; both keys remain governed exclusively by Eqs. (1)–(2), evaluated by the TPM against attestation outcomes alone. The manifest's classification and offset are advisory inputs to the OEM backend, which – upon confirming the report against its own records – generates a binary delta encoding only the implicated sectors and signs it through the unmodified channel of Paper 1. The resulting micro-patch is written through the target ECU's existing immutable boot stage rather than its application image, and must still pass the same signature verification and rollback check as any other update before installation. Consequently, an incorrect or adversarially influenced manifest can at most cause a wasted or misdirected patch attempt, which the existing integrity checks of Paper 1 would reject; it cannot itself release K_{Full}, K_{Limp}, or any unauthorized code path, since neither key's policy depends on anything the SLM produces.

4 Evaluation Methodology

As this report presents a design proposal, a live experimental implementation was not deployed. Instead, this section specifies a reproducible evaluation methodology designed to validate the claims of Section 3. To ensure directly comparable baselines, the proposed setup extends the prototype environment utilized by Plappert and Fuchs—specifically an SPI-attached *Infineon Iridium9670 TPM 2.0 (Linux) evaluation board* [7], [11], [12]. This hardware trust anchor is augmented with an *NXP S32G2 Vehicle Network Processor* [9]

central gateway and an embedded Edge NPU platform (e.g., *ARM Ethos-U65* [2]) inside the TCU to execute the quantized Small Language Model (SLM).

4.1 Measurement Instrumentation and Tooling

To collect high-fidelity, sub-millisecond metrics without introducing measurement overhead into the safety-critical control loop, we propose using the following specialized industrial tooling:

Vector CANoe/CANalyzer [17]: Connected directly to physical transceivers to manage passive frame logging and execute deterministic frame-level fault injections on the CAN/CAN-FD domain buses.

Saleae Logic Pro 16 [14]: A high-frequency digital logic analyzer hooked to physical SPI lines and the TPM's GPIO hardware interrupt pin to capture the raw, hardware-level latency of the real-time isolation loop.

TFLite Micro [5] **/ ONNX Runtime** [10] **Profiler**: Embedded profiling tools running directly on the TCU's NPU to track INT8 execution latencies and peak Static RAM (SRAM) memory footprint allocations out-of-band.

4.2 Mixed-Criticality Scenario Testing

To validate that the system correctly maintains driving safety while selectively preserving availability, the evaluation methodology forces two distinct operational test cases across the network:

Scenario A – Non-Critical Tier-$\mathcal{U}$ Failure: A software fault or cryptographic mismatch is injected into an auxiliary, non-critical ECU $j \in \mathcal{U}$ (e.g., the Infotainment Unit). The expected outcome is for the TPM to unseal the Limp-Home Authorization Key (K_{Limp}) via Eq. (2). The evaluation passes if Path 1 successfully blocks ECU_j's traffic and transitions the vehicle into a stable, degraded driving mode.

Scenario B – Critical Tier-$\mathcal{C}$ Failure: A cryptographic failure is simulated within a safety-critical ECU $i \in \mathcal{C}$ (e.g., the Powertrain Control Module). Because Eq. (2) requires all critical nodes to equal 1, neither driving key can be unsealed. The evaluation passes if the vehicle executes an absolute fail-secure response, refusing ignition and remaining completely immobilized.

4.3 Timing and Latency Metrics

To avoid the methodological error of implying that non-real-time AI diagnostic loops occur within live network timelines, Path 1 and Path 2 latencies must be measured and tracked as completely separate metrics:

- **Attestation Detection Latency (ΔT_{Detect}):** The raw time required for the TPM to evaluate the initial DICE challenge-response loop while the vehicle is stationary. This relies entirely on standard Paper 2 primitives; the original prototype reports Secure ECU Attestation times ranging from approximately 1.4 s (HMAC) to 3.1 s (RSA-PSS), with TPM processing – not transmission – as the dominant cost [11].
- **Isolation and Sub-Masking Reaction Latency ($\Delta T_{\text{Isolate}}$):** The real-time delta measured via logical analyzer probes from the exact microsecond the TPM sets $NV_{\text{Report}}[j] = 0$ to the moment the gateway clears its routing tables and injects its first baseline safe-state frame array. This path is expected to complete within a sub-millisecond timeline ($< 50\,\mu$s), running well beneath the standard $\sim$10 ms CAN timeout deadline.

TABLE 1
Proposed evaluation metrics for the two-path architecture

Metric	Method / Trigger	Expected Outcome
Scenario A – Tier-$\mathcal{U}$ failure	Fault injected into non-critical ECU $j \in \mathcal{U}$	K_{Limp} unsealed; Path 1 isolates ECU_j; vehicle enters degraded mode
Scenario B – Tier-$\mathcal{C}$ failure	Fault injected into safety-critical ECU $i \in \mathcal{C}$	Neither key unsealed; vehicle remains fail-secure and immobilized
Attestation detection latency (ΔT_{Detect})	TPM DICE challenge-response, vehicle stationary	$\sim$1.4–3.1 s (baseline, unmodified Paper 2)
Isolation reaction latency ($\Delta T_{\text{Isolate}}$)	Logic-analyzer probe, $NV_{\text{Report}}[j] = 0$ to first baseline frame	$< 50\,\mu$s ($\ll$10 ms CAN timeout)
SLM classification performance	Synthetic traces: single-bit faults vs. multi-byte tamper + UDS `0x10`/`0x28`	TPR, FPR, F1-score reported
Bounded-failure regression	Adversarial/malformed manifests fed to TCU	No change to gateway routing state; invalid offsets rejected by signature check
Six-attack security regression	Rollback, replay, TCU compromise, in-motion update, mix-and-match, wrong-target	No new vector; `IsolateECU` cannot be spoofed against healthy/critical nodes

4.4 SLM Performance and Bounded Failure Regression

The final evaluation layer assesses model accuracy, containment boundaries, and security regression vectors out-of-band. First, we propose profiling the SLM's classification performance (TPR, FPR, and F1-score) using synthetic diagnostic traces to differentiate between transient hardware faults (isolated single-bit parity errors) and active firmware tampering (multi-byte modifications paired with anomalous UDS service `0x10` or `0x28` frame floods). Second, to validate safety-by-construction bounding, the TCU is subjected to adversarial diagnostic inputs; the evaluation passes if malformed manifests fail to alter the gateway's real-time routing state and if invalid memory offsets are caught by the TPM's unmodified signature verification layer during the micro-patch cycle. Finally, to ensure no new vulnerabilities or self-inflicted Denial of Service vectors are introduced, the protocol mandates re-running the six core network attack scenarios from the original literature (rollback, replay, TCU compromise, in-motion updates, mix-and-match, and wrong-target spoofing), confirming that an unauthorized node cannot maliciously execute or spoof the `IsolateECU` mechanism to knock healthy, critical nodes offline.

5 Related Work

5.1 TPM/DICE-based update security

This report builds directly on the two-paper system of Plappert and Fuchs. Paper 1 [12] establishes the TPM-anchored authenticated distribution and installation authorization mechanisms that the proposed micro-patch delivery reuses unmodified. Paper 2 [11] establishes the DICE-based attestation and NVReport/KSP mechanism that Path 1 extends. As established in Section 1, both papers exclude DoS by design, [11], which is precisely the gap this report targets; our contribution is therefore best understood as a deployment-layer extension of an already-validated cryptographic design, not a replacement for it.

5.2 CAN bus intrusion response and isolation

Hardware-triggered isolation of a misbehaving ECU has been studied previously. Sagong et al. propose a voltage-based intrusion detection and response scheme that physically disconnects a node from the CAN bus once an anomalous electrical signature is observed on the wire [13]. This line of work differs from Path 1 in its trigger: voltage-based schemes detect anomalies at the physical layer in real time during normal operation, whereas Path 1 is triggered by a cryptographic attestation outcome during the OTA update window. The two approaches are complementary rather than competing.

5.3 ECU firmware recovery

Dafoe et al. present a framework for real-time restoration of compromised ECU firmware, combining intrusion detection with a TrustZone-based recovery mechanism and a flash translation layer that allows safe re-flashing without full physical replacement [4]. This is the closest prior work to the remediation half of our proposal, but it assumes a TrustZone-based trust anchor, is triggered by IDS alerts rather than the TPM/DICE chain used here, and does not include a diagnostic characterization step analogous to the Remediation Manifest – it targets re-flashing once a compromise is already assumed, rather than first distinguishing fault classes to enable a targeted micro-patch.

5.4 Scalable and collective remote attestation

A separate line of work addresses the cost of attesting many devices, rather than the response to a failure. SEDA introduced collective attestation for large device swarms, distributing the verification burden across the swarm itself [3]. SANA extends this with aggregate signatures and explicitly considers DoS resilience *of the attestation protocol itself* [1]. These works solve a different problem than this report: they reduce the cost of verifying many devices through topology-based aggregation, whereas our proposal assumes the existing centralized TPM verifier of Paper 2 and instead defines what should happen to vehicle availability and network exposure once a verification outcome is known for a specific ECU.

TABLE 2
Positioning of the proposed architecture relative to related work

Work	Trigger	Response	Anchor
Plappert & Fuchs [11], [12]	Signed manifest / DICE attestation outcome	All-or-nothing drive-mode lock (no isolation, no recovery)	TPM 2.0 + DICE
Sagong et al. [13]	Voltage-domain physical-layer anomaly	Physical bus disconnect of misbehaving node	None (analog signal)
Dafoe et al. [4]	IDS alert on CAN traffic	TrustZone re-flash via flash translation layer	ARM TrustZone
SEDA [3] / SANA [1]	N/A – verification cost, not failure response	Swarm-distributed aggregate attestation	Distributed (no single TPM)
LLM/SLM in-vehicle work [6], [8], [15]	CAN telemetry anomaly, real time	Direct anomaly classification in the live control path	None (no trust anchor)
This work	TPM/DICE attestation failure during OTA window	Tiered gateway isolation (Path 1) + advisory SLM diagnosis (Path 2)	TPM 2.0 + DICE (reused)

5.5 Language models in automotive and embedded security

The use of language models for cybersecurity tasks in vehicular and embedded contexts is an emerging but still nascent area. Recent work has explored LLM-driven threat intelligence for zero-day attack detection in electric vehicle cyber-physical systems from CAN telemetry [15], and pretrained foundation models fitted to decoded CAN signals for downstream tasks such as driver risk modeling and predictive maintenance [6]. Separately, small, resource-constrained language models have been optimized for direct on-device deployment in vehicle head units, using pruning, healing, and quantization to achieve real-time function-calling performance under automotive hardware constraints [8]. To the best of our knowledge, no prior work combines an on-device SLM with a TPM/DICE attestation chain as an *out-of-band, advisory-only* diagnostic layer structurally prevented from influencing the real-time control path or the cryptographic authorization chain – existing work either applies language models directly to real-time anomaly detection, inheriting the latency and reliability concerns this report avoids by confining the SLM to the asynchronous path, or considers automotive language-model deployment independently of an attestation-based trust anchor and outside the security domain altogether.

5.6 Summary of positioning

Table 2 summarizes how each line of related work differs from the proposed architecture along the dimensions of trigger mechanism, response type, and trust anchor.

6 Conclusions

While the cryptographic frameworks of Plappert and Fuchs provide robust security, their all-or-nothing attestation poli-

cies introduce a critical availability gap by freezing the vehicle over a single non-critical node failure. This report proposed a two-path resilience architecture comprising an immediate, deterministic path (Path 1) and an asynchronous, diagnostic path (Path 2) to resolve this vulnerability by decoupling real-time physical availability from out-of-band diagnostics. By completely isolating the diagnostic Small Language Model (SLM) from the active driving loop and Trusted Platform Module (TPM) authorization policies, the system achieves a safety-by-construction design where algorithmic faults are strictly bounded. Future work will focus on expanding the hardware fault signature dataset, integrating complementary CAN intrusion detection triggers [13], and optimizing network tiering to mitigate scalability limitations in high-density ECU architectures. Additionally, the framework can be extended to address full system availability during safety-critical ($\mathcal{C}$) ECU failures; while the current architecture intentionally defaults to a secure locked state to protect vehicle physics, future iterations will investigate safe degradation layers—such as secondary backup hardware handovers or functional physical bypasses—to safely mitigate critical-tier faults without triggering a total ignition lockout.

REFERENCES

[1] M. Ambrosin, M. Conti, A. Ibrahim, G. Neven, A.-R. Sadeghi, and M. Schunter, "Sana: secure and scalable aggregate network attestation," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 2016, pp. 731–742.

[2] Arm Limited, "Ethos-U65 NPU Technical Reference Manual," https://developer.arm.com/Processors/Ethos-U65, 2024.

[3] N. Asokan, F. Brasser, A. Ibrahim, A.-R. Sadeghi, M. Schunter, G. Tsudik, and C. Wachsmann, "Seda: Scalable embedded device attestation," in *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 2015, pp. 964–975.

[4] J. Dafoe, H. Singh, N. Chen, and B. Chen, "Enabling real-time restoration of compromised ecu firmware in connected and autonomous vehicles," in *International Conference on Security and Privacy in Cyber-Physical Systems and Smart Vehicles*. Springer, 2023, pp. 15–33.

[5] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang *et al.*, "Tensorflow lite micro: Embedded machine learning for tinyml systems," *Proceedings of machine learning and systems*, vol. 3, pp. 800–811, 2021.

[6] A. Esashi, P. Lertpongrujikorn, J. Makino, Y. Fujimoto, and M. A. Salehi, "Foundation can lm: A pretrained language model for automotive can data," *arXiv preprint arXiv:2602.00866*, 2026.

[7] Infineon Technologies AG, "Iridium9670 tpm2.0 linux," https://www.infineon.com/cms/de/product/evaluation-boards/iridium9670-tpm2.0-linux/, 2023, online. Last accessed: 2023-09-22.

[8] Y. S. Khiabani, F. Atif, C. Hsu, S. Stahlmann, T. Michels, S. Kramer, B. Heidrich, M. S. Sarfraz, J. Merten, and F. Tafazzoli, "Optimizing small language models for in-vehicle function-calling," *arXiv preprint arXiv:2501.02342*, 2025.

[9] NXP Semiconductors, "S32G2 Vehicle Network Processor Data Sheet," https://www.nxp.com/products/S32G2, 2024.

[10] ONNX Runtime developers, "ONNX Runtime," https://onnxruntime.ai/, 2024.

[11] C. Plappert and A. Fuchs, "Secure and lightweight ecu attestations for resilient over-the-air updates in connected vehicles," in *Proceedings of the 39th Annual Computer Security Applications Conference*, 2023, pp. 283–297.

[12] ——, "Secure and lightweight over-the-air software update distribution for connected vehicles," in *Proceedings of the 39th Annual Computer Security Applications Conference*, 2023, pp. 268–282.

[13] S. U. Sagong, R. Poovendran, and L. Bushnell, "Mitigating vulnerabilities of voltage-based intrusion detection systems in controller area networks," *arXiv preprint arXiv:1907.10783*, 2019.

[14] Saleae Inc., "Logic Pro 16 – 16-Channel USB Logic Analyzer Datasheet," https://www.saleae.com/products/logic-pro-16, 2023.

[15] A. Tirulo, S. Chauhan, and M. Shafie-khah, "Llm-powered threat intelligence: Proactive detection of zero-day attacks in electric vehicle cyber-physical systems," *Sustainable Energy, Grids and Networks*, p. 101877, 2025.

[16] United Nations Economic Commission for Europe, "Un regulation no. 155 – cyber security and cyber security management system; un regulation no. 156 – software update and software update management system," UNECE WP.29, 2021.

[17] Vector Informatik GmbH, "CANoe / CANalyzer – ECU and Network Testing," https://www.vector.com/int/en/products/products-a-z/c10165, 2024.